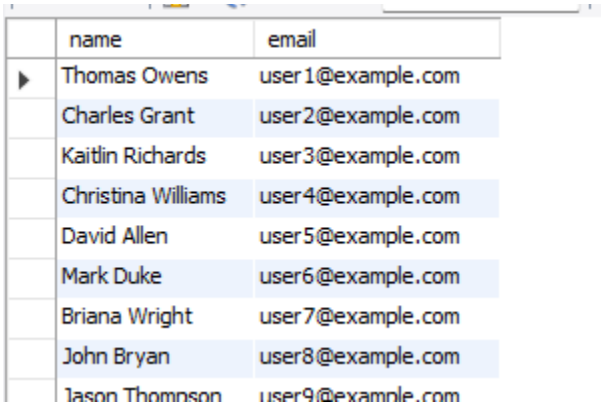
Questions

Level 1: Basics

1.**Retrieve customer names and emails for email marketing.**

SELECT name, email FROM customers;



name	email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com

2.**View complete product catalog with all available details**

SELECT *FROM products;

product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57

3. List all unique product categories

SELECT DISTINCT category FROM products;

category
Home
Clothing
Electronics
Toys
Books

4. Show all products priced above ₹1,000

SELECT * FROM products WHERE price > 1000;

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38

5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)

SELECT * FROM products WHERE price BETWEEN 2000 AND 5000;

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55

6. Fetch data for specific customer IDs (e.g., from loyalty program list)SELECT * FROM customers WHERE customer_id IN (1, 5, 10, 15

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
NULL	NULL	NULL	NULL	NULL

7. Identify customers whose names start with the letter ‘A’

SELECT * FROM customers WHERE name LIKE 'A%';

customer_id	name	email	phone	created_at
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
NULL	NULL	NULL	NULL	NULL

8. List electronics products priced under ₹3,000

SELECT * FROM products WHERE category = 'Electronics' AND price < 3000;

product_id	name	category	price	stock_quantity	added_on
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38
NULL	NULL	NULL	NULL	NULL	NULL

9. Display product names and prices in descending order of price

SELECT name, price FROM products ORDER BY price DESC;

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66

10. Display product names and prices, sorted by price and then by name

SELECT name, price FROM products ORDER BY price DESC, name ASC;

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66

Level 2: Filtering and Formatting

1.Retrieve orders where customer information is missing (possibly due to data migration or deletion)

```
SELECT * FROM orders WHERE customer_id IS NULL;
```

order_id	customer_id	order_date	status	total_amount
NULL	NULL	NULL	NULL	NULL

2.Display customer names and emails using column aliases for frontend readability

```
SELECT name AS Customer_Name, email AS Email_Address FROM customers;
```

Customer_Name	Email_Address
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com

3. Calculate total value per item ordered by multiplying quantity and item price

```
SELECT order_item_id, quantity, item_price, quantity * item_price AS Total_Value FROM order_items;
```

order_item_id	quantity	item_price	Total_Value
1	2	4707.14	9414.28
2	3	177.40	532.20
3	3	985.19	2955.57
4	1	2208.99	2208.99
5	2	4734.89	9469.78
6	1	4707.14	4707.14
7	2	4897.36	9794.72
8	3	4897.36	14692.08
9	1	1429.45	1429.45
10	1	723.97	723.97

4. Combine customer name and phone number in a single column

SELECT CONCAT(name,' - ',phone) AS Customer_Contact FROM customers;

Customer_Contact
Thomas Owens - 142-479-1945
Charles Grant - 9153947511
Kaitlin Richards - 2073473421
Christina Williams - 586-605-5061x06
David Allen - (751)456-8289x1
Mark Duke - (144)957-2811
Briana Wright - 223-833-9635
John Bryan - 045.568.0798x27
Jason Thompson - 1862659420
Shawn Hill - (268)113-3152x7

5. Extract only the date part from order timestamps for date-wise reporting

SELECT order_id, DATE(order_date) AS Order_Date FROM orders;

order_id	Order_Date
1	2025-03-02
2	2024-10-09
3	2025-05-08
4	2024-09-19
5	2025-04-08
6	2024-10-25
7	2024-07-29
8	2024-07-30
9	2025-06-10
10	2025-02-16

6. List products that do not have any stock left

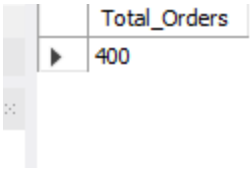
SELECT * FROM products WHERE stock_quantity = 0;



Level 3: Aggregations

1.Count the total number of orders placed

SELECT COUNT(*) AS Total_Orders FROM orders;



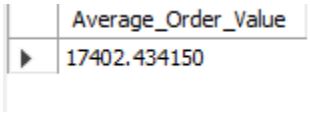
2. Calculate the total revenue collected from all orders

SELECT SUM(total_amount) AS Total_Revenue FROM orders;



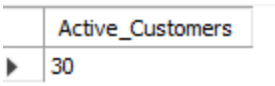
3. Calculate the average order value

SELECT AVG(total_amount) AS Average_Order_Value FROM orders;



4. Count the number of customers who have placed at least one order

SELECT COUNT(DISTINCT customer_id) AS Active_Customers FROM orders;



5. Find the number of orders placed by each customer

SELECT customer_id, COUNT(order_id) AS Total_Orders FROM orders GROUP BY customer_id;

customer_id	Total_Orders
1	12
2	17
3	17
4	9
5	14
6	16
7	13
8	10
9	10

6. Find total sales amount made by each customer

SELECT customer_id, SUM(total_amount) AS Total_Sales FROM orders GROUP BY customer_id;

customer_id	Total_Sales
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78
9	106226.03
10	252722.75

7. List the number of products sold per category

SELECT p.category, SUM(oi.quantity) AS Products_Sold FROM products p JOIN order_items oi ON p.product_id = oi.product_id GROUP BY p.category;

category	Products_Sold
Home	443
Clothing	559
Electronics	687
Toys	405
Books	350

8. Find the average item price per category

SELECT category, AVG(price) AS Average_Price FROM products GROUP BY category;

category	Average_Price
Home	2146.367500
Clothing	3434.581538
Electronics	2653.388571
Toys	2516.526250
Books	3167.084286

9. Show number of orders placed per day

SELECT DATE(order_date) AS Order_Day, COUNT(*) AS Total_Orders FROM orders GROUP BY DATE(order_date) ORDER BY Order_Day;

Order_Day	Total_Orders
2024-06-15	1
2024-06-18	1
2024-06-19	3
2024-06-20	2
2024-06-21	2
2024-06-23	2
2024-06-24	1
2024-06-25	1
2024-06-30	2
2024-07-03	1

10. List total payments received per payment method

SELECT method, SUM(amount_paid) AS Total_Payments FROM payments GROUP BY method;

method	Total_Payments
Credit Card	1754603.10
Net Banking	1658383.90
UPI	1617408.78
Debit Card	1930577.88

Level 4: Multi-Table Queries (JOINS)

1.Retrieve order details along with the customer name (INNER JOIN)

SELECT o.order_id, c.name, o.order_date, o.total_amount FROM orders o INNER JOIN customers c
ON o.customer_id = c.customer_id;

order_id	name	order_date	total_amount
14	Thomas Owens	2024-09-24 21:21:38	15803.34
17	Thomas Owens	2024-08-19 21:17:57	11173.77
61	Thomas Owens	2024-12-26 23:21:58	13053.00
76	Thomas Owens	2025-01-14 22:59:54	23506.81
92	Thomas Owens	2024-09-26 11:33:58	834.75
109	Thomas Owens	2025-03-17 04:56:18	12190.14
127	Thomas Owens	2025-06-10 18:01:32	20160.64
135	Thomas Owens	2025-03-11 22:58:23	8891.79
144	Thomas Owens	2024-09-19 09:18:22	12093.54
221	Thomas Owens	2024-09-25 05:49:39	12437.61

2. Get list of products that have been sold (INNER JOIN with order_items)

SELECT DISTINCT p.product_id,p.name FROM products p INNER JOIN order_items oi ON p.product_id = oi.product_id;

product_id	name
1	Plant No
2	Population Social
3	Available Answer
4	Any Question
5	Natural Network
6	If Whatever
7	Response Indeed
8	Every Amount
9	Common Study
10	Development System

3. List all orders with their payment method (INNER JOIN)

SELECT o.order_id, p.method, p.amount_paid FROM orders o INNER JOIN payments p ON o.order_id = p.order_id;

order_id	method	amount_paid
14	UPI	15803.34
17	UPI	11173.77
61	Net Banking	13053.00
76	Credit Card	23506.81
92	Net Banking	834.75
109	UPI	12190.14
127	Debit Card	20160.64
135	Debit Card	8891.79
144	UPI	12093.54
221	UPI	12437.61

4. Get list of customers and their orders (LEFT JOIN)

SELECT c.customer_id, c.name, o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;

customer_id	name	order_id
1	Thomas Owens	14
1	Thomas Owens	17
1	Thomas Owens	61
1	Thomas Owens	76
1	Thomas Owens	92
1	Thomas Owens	109
1	Thomas Owens	127
1	Thomas Owens	135
1	Thomas Owens	144
1	Thomas Owens	221

5. List all products along with order item quantity (LEFT JOIN)

SELECT p.product_id, p.name, oi.quantity FROM products p LEFT JOIN order_items oi ON p.product_id = oi.product_id;

product_id	name	quantity
1	Plant No	1
1	Plant No	3
1	Plant No	1
1	Plant No	1
1	Plant No	2
1	Plant No	3
1	Plant No	2
1	Plant No	2
1	Plant No	2
1	Plant No	2

6. List all payments including those with no matching orders (RIGHT JOIN)

SELECT o.order_id, p.payment_id, p.amount_paid FROM orders o RIGHT JOIN payments p ON o.order_id = p.order_id;

order_id	payment_id	amount_paid
1	1	9414.28
2	2	532.20
3	3	5164.56
4	4	9469.78
5	5	14501.86
6	6	31050.17
7	7	3043.67
8	8	32714.06
9	9	24219.20
10	10	24342.52

7. Combine data from three tables: customer, order, and payment

SELECT c.name,o.order_id, o.total_amount, p.method, p.amount_paid FROM customers c JOIN orders o ON c.customer_id = o.customer_id JOIN payments p ON o.order_id = p.order_id;

name	order_id	total_amount	method	amount_paid
Thomas Owens	14	15803.34	UPI	15803.34
Thomas Owens	17	11173.77	UPI	11173.77
Thomas Owens	61	13053.00	Net Banking	13053.00
Thomas Owens	76	23506.81	Credit Card	23506.81
Thomas Owens	92	834.75	Net Banking	834.75
Thomas Owens	109	12190.14	UPI	12190.14
Thomas Owens	127	20160.64	Debit Card	20160.64
Thomas Owens	135	8891.79	Debit Card	8891.79
Thomas Owens	144	12093.54	UPI	12093.54
Thomas Owens	221	12437.61	UPI	12437.61

Level 5: Subqueries (Inner Queries)

1.**List all products priced above the average product price**

SELECT * FROM products WHERE price >(SELECT AVG(price)FROM products);

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27

products 37 ×

2. Find customers who have placed at least one order

SELECT * FROM customers WHERE customer_id IN (SELECT customer_id FROM orders);

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43

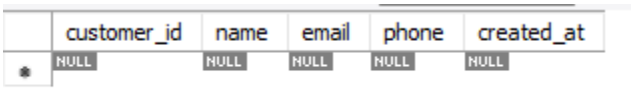
3. Show orders whose total amount is above the average for that customer

SELECT *FROM orders o WHERE total_amount > (SELECT AVG(total_amount) FROM orders WHERE customer_id = o.customer_id);

order_id	customer_id	order_date	status	total_amount
6	29	2024-10-25 07:33:59	Cancelled	31050.17
8	19	2024-07-30 22:49:49	Cancelled	32714.06
9	6	2025-06-10 17:00:25	Pending	24219.20
10	28	2025-02-16 12:45:59	Delivered	24342.52
14	1	2024-09-24 21:21:38	Shipped	15803.34
16	5	2024-09-09 03:00:40	Shipped	22856.73
18	13	2024-06-15 12:57:04	Cancelled	32001.24
21	22	2025-03-12 18:53:46	Pending	25364.11
22	20	2025-02-21 11:21:06	Shipped	21281.44
24	6	2024-06-19 07:09:04	Pending	22882.19

4. Display customers who haven't placed any orders

SELECT *FROM customers WHERE customer_id NOT IN (SELECT customer_id FROM orders);



customer_id	name	email	phone	created_at
NULL	NULL	NULL	NULL	NULL

5. Show products that were never ordered

SELECT *FROM products WHERE product_id NOT IN (SELECT product_id FROM order_items);



product_id	name	category	price	stock_quantity	added_on
NULL	NULL	NULL	NULL	NULL	NULL

6. Show highest value order per customer

SELECT *FROM orders o WHERE total_amount =(SELECT MAX(total_amount) FROM orders WHERE customer_id = o.customer_id);

order_id	customer_id	order_date	status	total_amount
18	13	2024-06-15 12:57:04	Cancelled	32001.24
33	8	2025-04-27 01:26:11	Pending	36147.09
62	10	2025-06-13 23:21:01	Cancelled	35723.17
66	27	2024-06-19 09:41:16	Pending	26176.05
68	24	2025-03-15 07:27:20	Delivered	36159.92
80	12	2025-03-28 18:42:12	Delivered	38656.26
107	14	2025-02-01 02:02:18	Shipped	48042.06
121	16	2024-11-21 16:50:01	Pending	37415.67
128	7	2025-05-05 16:49:18	Pending	28589.04
133	11	2024-11-20 00:44:46	Shipped	37129.82

7. Highest Order Per Customer (Including Names)

SELECT c.customer_id, c.name, o.order_id, o.total_amount FROM customers c JOIN orders o ON c.customer_id = o.customer_id WHERE o.total_amount =(SELECT MAX(total_amount) FROM orders WHERE customer_id = c.customer_id);

customer_id	name	order_id	total_amount
1	Thomas Owens	278	32015.16
2	Charles Grant	315	42056.04
3	Kaitlin Richards	281	41679.11
4	Christina Williams	386	25747.34
5	David Allen	266	39921.78
6	Mark Duke	330	39003.19
7	Briana Wright	128	28589.04
8	John Bryan	33	36147.09
9	Jason Thompson	348	21414.44
10	Shawn Hill	62	35723.17

Level 6: Set Operations

1.List all customers who have either placed an order or written a product review

SELECT customer_id FROM orders UNION SELECT customer_id FROM product_reviews;

customer_id
1
2
3
4
5
6
7
8
9
10

2.List all customers who have placed an order as well as reviewed a product [intersect not supported]

SELECT DISTINCT customer_id FROM orders WHERE customer_id IN (SELECT customer_id FROM product_reviews);

customer_id
1
2
4
6
7
9
10
11
13
14

orders 45